Question 1

Answer saved

Marked out of 4.00

Remove flag

You design a standard 2-layer fully connected neural network. All activations are sigmoids and your optimizer is stochastic gradient descent. You initialize all the weights and biases to zero and forward propagate an input $\mathbf{x}$ in the network. What is the output $\hat{y}$ of the network?

Select one:

- a. 0
- b. $e^{\mathbf{x}}$
- c. 0.5
- d. -1
- e. 1

- **2-layer** fully connected neural network
- **Sigmoid** activations
- **Weights and biases initialized to 0**
- **Optimizer and training don't matter yet — we only do a forward pass**
- Input: some vector x

🔁 Forward pass:

1 First layer pre-activation:

$$z^{(1)} = W^{(1)}x + b^{(1)} = 0 \cdot x + 0 = 0$$

2 First layer activation (sigmoid):

$$a^{(1)} = \sigma(z^{(1)}) = \sigma(0) = \frac{1}{1+e^{-0}} = 0.5$$

3 Second layer pre-activation:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 0 \cdot 0.5 + 0 = 0$$

4 Second layer activation (sigmoid again):

$$a^{(2)} = \sigma(z^{(2)}) = \sigma(0) = 0.5$$

If all weights are initialized to zero, **every neuron computes the same output**, and all gradients become the same — leading to **no learning** (symmetry breaking fails). That's why modern networks use random initialization (e.g. Xavier, He).

Question **2**

Answer saved

Marked out of 4.00

Remove flag

Consider the model defined in the question before with parameters initialized with zeros. $\mathbf{W}^{[2]}$ denotes the weight matrix of the second layer. You forward propagate a batch of examples, and then backpropagate the gradients and update the parameters. Which of the following statements is true?

Select one:

- a. The entries of the weight matrix $\mathbf{W}^{[2]}$ are all positive.
- b. The entries of the weight matrix $\mathbf{W}^{[2]}$ may be positive or negative.
- c. The entries of the weight matrix $\mathbf{W}^{[2]}$ are all negative.
- d. The entries of the weight matrix $\mathbf{W}^{[2]}$ are all zeros.

Clear my choice

I think b) should be correct. I think not all zeros, unless mean target = 0.5. But not sure.

Question **5**

Answer saved

Marked out of 6.00

Remove flag

You are given an input $\mathbf{x} = \begin{pmatrix} 1 & -2 \\ -1 & 2 \end{pmatrix}$ and a kernel weight matrix $\mathbf{w} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$. What is the result of the **transpose convolution operation** $*^T$ (stride 2) of

$$\mathbf{w} *^T \mathbf{x} = ?$$

Select one:

- a. $= \begin{pmatrix} 0 & 1 & -1 \\ 2 & -1 & -1 \\ -4 & -2 & 6 \end{pmatrix}$
- b. $= \begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix}$
- c. $= \begin{pmatrix} 0 & 1 & 0 & -2 \\ 2 & 3 & -4 & -6 \\ 0 & -1 & 0 & 2 \\ 2 & 3 & -4 & -6 \end{pmatrix}$
- d. $= \begin{pmatrix} 0 & 2 & 1 \\ 6 & -14 & 6 \\ 2 & 3 & 0 \end{pmatrix}$
- e. $= \begin{pmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & -2 & -2 \\ 2 & 2 & 4 & 4 \\ 2 & 2 & -4 & -4 \end{pmatrix}$
- f. $= (3)$
- g. $= (42)$
- h. $= \begin{pmatrix} 8 & -5 & 2 \\ -11 & 5 & 13 \\ 19 & 21 & -29 \end{pmatrix}$

Clear my choice

Question 15

Answer saved

Marked out of 9.00

Remove flag

Suppose that you have a fixed, multi-dimentional, non-Gaussian distribution $p(\boldsymbol{x})$ that you want to approximate with a Gaussian distribution. You decide that you want to approximate this distribution with a Gaussian $q(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with some mean vector $\boldsymbol{\mu}$ and some covariance matrix $\boldsymbol{\Sigma}$. You now decide to minimize the Kullback-Leibler divergence between the two distributions $\mathrm{KL}(p \,||\, q)$ with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$.

Which expressions for $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ mimimize the KL-divergence between the two distributions?

(The usual notation of the lecture is used: $\mathbb{E}$ is expectation, $\mathbb{C}\mathrm{ov}$ is the covariance, $\boldsymbol{I}$ is the identity matrix)

- [ ] a. $\boldsymbol{\mu} = \mathbb{E}(\boldsymbol{x}^T\boldsymbol{x})$ and $\boldsymbol{\Sigma} = \boldsymbol{I}$
- [x] b. $\boldsymbol{\mu} = \mathbb{E}(\boldsymbol{x})$ and $\boldsymbol{\Sigma} = \mathbb{C}\mathrm{ov}(\boldsymbol{x})$
- [ ] c. $\boldsymbol{\mu} = \mathbb{E}(\boldsymbol{x}^T\boldsymbol{x})$ and $\boldsymbol{\Sigma} = \mathbb{E}(\boldsymbol{x}^T)$
- [ ] d. $\boldsymbol{\mu} = \mathbb{E}(\boldsymbol{x})$ and $\boldsymbol{\Sigma} = \boldsymbol{I}$
- [x] e. $\boldsymbol{\mu} = \mathbb{E}(\boldsymbol{x})$ and $\boldsymbol{\Sigma} = \mathbb{E}(\boldsymbol{x}\boldsymbol{x}^T)$

After taking a look at gpt solution and previous discussions in the chat I think e) should be wrong and only b) correct.

❌ Why the other marked answer (e) is **wrong**:

❌ e. $\mu = \mathbb{E}[x]$, $\Sigma = \mathbb{E}[xx^T]$

- $\mathbb{E}[xx^T]$ is the **second moment**, **not** the **covariance**.
- Covariance is:

$$\mathrm{Cov}(x) = \mathbb{E}[xx^T] - \mathbb{E}[x]\mathbb{E}[x]^T$$

- So **option e** overestimates the covariance and gives the wrong spread.

Question 21

Answer saved

Marked out of 6.00

Remove flag

The main difference between the frequentist and Bayesian approach is marginalization over parameters when we define the probability for a class label given the input $\mathbf{x}$ and dataset $\mathcal{D}$. To give a more formal definition:

$$p(\mathbf{y}^{test}|\mathbf{x}^{test}, \mathcal{D}) = \mathbb{E}_{\mathbf{w}\sim p(\mathbf{w}|\mathcal{D})}\left[p(\mathbf{y}^{test}|\mathbf{w}, \mathbf{x}^{test})\right]$$

Deep Ensembles try to approximate the full expectation with (weighted) averages. Which of this options are valid?

$\mathbb{E}_{\mathbf{w}\sim p(\mathbf{w}|\mathcal{D})}\left[p(\mathbf{y}^{test}|\mathbf{w}, \mathbf{x}^{test})\right] \approx$

- [x] a. $\approx \frac{1}{M}\sum_{m=1}^{M} p(\mathbf{y}^{test}|\mathbf{w}_m, \mathbf{x}^{test})$, where $\mathbf{w}_m$ denotes the weights of the m-th network of the ensemble.
- [ ] b. $\approx p(\mathbf{y}^{test}|\mathbf{w}, \mathbf{x}^{test})$
- [x] c. $\approx \frac{1}{M}\sum_{m=1}^{M} \alpha_m p(\mathbf{y}^{test}|\mathbf{w}_m, \mathbf{x}^{test})$ with $\alpha_m = \mathrm{softmax}(\mathbf{z})_m$ for some random normal distributed vector $\mathbf{z}$
- [x] d. $\approx \frac{1}{M}\sum_{m=1}^{M} p(\mathbf{y}_m^{test}|\mathbf{w}, \mathbf{x}_m^{test})$ where the subscript m of the test data denotes the m-th equally sized fraction of the full data.
- [ ] e. $\approx \sum_{m=1}^{M} p(\mathbf{y}_m^{test}|\mathbf{w}, \mathbf{x}_m^{test})$ where the subscript m of the test data denotes the m-th equally sized fraction of the full data.
- [ ] f. $\approx \sum_{m=1}^{M} p(\mathbf{y}^{test}|\mathbf{w}_m, \mathbf{x}^{test})$

c) also probably also correct

15.1. Deep Ensembles

- Main idea: approximate expectation with averages

$$p(\boldsymbol{y}^{\text{test}} \mid \boldsymbol{x}^{\text{test}}, \mathcal{D}) = \mathbb{E}_{\boldsymbol{w} \sim p(\boldsymbol{w}|\mathcal{D})} \left[p(\boldsymbol{y}^{\text{test}} \mid \boldsymbol{w}, \boldsymbol{x}^{\text{test}}) \right]$$

- Train several DNNs with different random seeds, obtain parameters $\boldsymbol{w}^{(m)}$ and take average

$$p(\boldsymbol{y}^{\text{test}} \mid \boldsymbol{x}^{\text{test}}, \mathcal{D}) \approx \frac{1}{M} \sum_{m=1}^{M} p(\boldsymbol{y}^{\text{test}} \mid \boldsymbol{w}^{(m)}, \boldsymbol{x}^{\text{test}},)$$

- Averages can be weighted differently
- Appears as simple technique, but performs well in practice

Question 22

Answer saved

Marked out of 6.00

Remove flag

Assume that you want to implement a new layer for a deep neural network that employs the following transformation

$$\boldsymbol{h}^{(l+1)} = \underbrace{\boldsymbol{M}\text{softmax}(\beta\, \boldsymbol{M}^T\, \boldsymbol{h}^{(l)})}_{:=\boldsymbol{p}},$$

where $\boldsymbol{h}^{(l)} \in \mathbb{R}^d$ is the input to this layer l, β is a positive hyperparameter to be set by the user, $\boldsymbol{M} \in \mathbb{R}^{d \times J}$ is a matrix of J patterns with d features provided by the domain expert, and $\boldsymbol{p}$ is defined as $\boldsymbol{p} := \text{softmax}(\beta\, \boldsymbol{M}^T\, \boldsymbol{h}^{(l)})$. Which of the following statements about this layer are true?

- [x] a. $\boldsymbol{h}^{(l+1)}$ is equivariant to permutations of the columns of $\boldsymbol{M}$.
- [x] b. $\boldsymbol{p}$ is invariant to permutations of the columns of $\boldsymbol{M}$.
- [ ] c. $\boldsymbol{p}$ is invariant to permutations of the columns of $\boldsymbol{M}$.
- [ ] d. $\boldsymbol{p}$ is equivariant to permutations of the columns of $\boldsymbol{M}$.
- [x] e. $\boldsymbol{p}$ is equivariant to permutations of the rows of $\boldsymbol{M}$.
- [ ] f. $\boldsymbol{h}^{(l+1)}$ is invariant to permutations of the columns of $\boldsymbol{M}$.
- [ ] g. This layer cannot be built in deep learning architectures because gradients, i.e. $\frac{\partial \boldsymbol{h}^{(l+1)}}{\partial \boldsymbol{h}^{(l)}}$, cannot be backpropagated

22 d and f correct.

d) Matrix multiplication is equivariant to the permutation of rows of the first matrix. But M is transposed. Softmax doesn't change the order.

f) It is invariant because you change the order of columns of M and the order of rows of p changes accordingly so in the end you have the same result

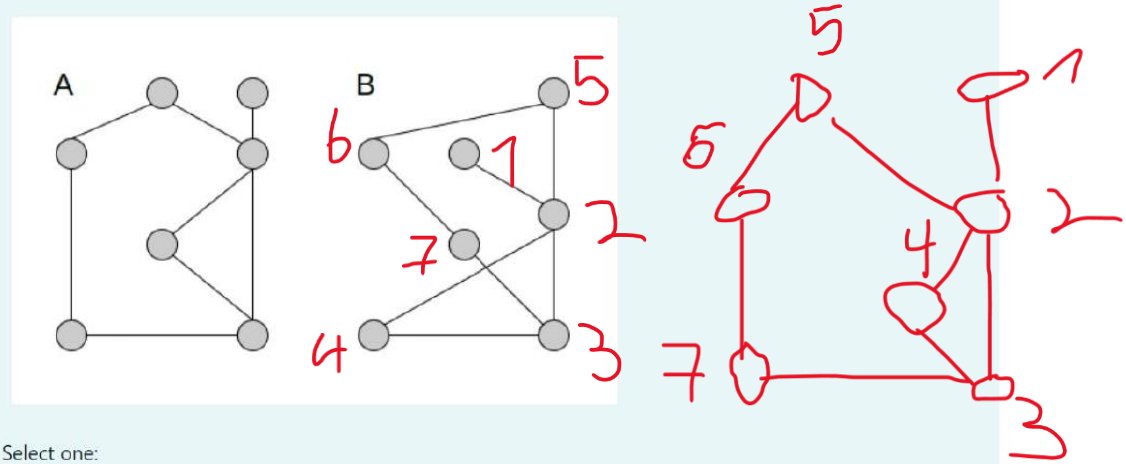
Question 23

Answer saved

Marked out of 3.00

Flag question

Do the following two graphs fulfill the necessary condition for graph isomorphism, the Weisfeiler-Lehman test?



Select one:

- True
- False